

actual program

1) $\Rightarrow a = 10$

$b = 20$

$c = a + b$

print (c)

interview X

2) Print ("Enter a value 1")

$a = \text{Input}()$

Print ("Enter a value 2")

$b = \text{Input}()$

$c = a + b$

Print () $\rightarrow$ output (1020)

✓ 3) Print ("Enter a value 1")

$a = \text{Int}(\text{Input}())$

Print ("Enter a value 2")

$b = \text{Int}(\text{Input}())$

$c = a + b$

Print ()

or

$a = \text{int}(\text{Input}("Enter a value 1"))$

Conditional statement

1. if

2. elif

3. else

4. nested if

```
print ("Enter a value 1")
```

```
a = int(input())
```

```
print ("Enter a value 2")
```

```
if (a > b):
```

```
    print ("a is greater")
```

```
elif (b > a):
```

```
    print ("b is greater")
```

```
else:
```

```
    print ("a == b")
```

next if (a > b):

if (a > c):

```
    print ("a is greater")
```

Looping statement

1) for loop

2) while loop

1) for loop

Syn:- for var-name in element 1, element 2, ---

↓ →
 ↳ block of code

remaining stmt.

ex:- for i in 1, 2, 5, 10:
 print(i)

* for i in "Ravi", "Sneha", "Darsan":

```
    print(i)
```

o/p < Ravi

Sneha

Darsan.

Date: / /

str = "sneha"

for i in str:

print(i, end=" ") o/p: sneha

or

print(i) -> (we) del

↓
a

Range function in for loop

1) range(stop)

2) range(start, stop)

3) range(start, stop, step)

syn :- for var_name in range(stop_value):

↓ ≡ & logic

remaining stmt

ex 1 :- for i in range(5):

print("heis")

o/p: heis

heis

⋮

end value 5

i ⇒ 0 1 2 3 4 X

0 1 X 2 X 3 X 4

or

for i in range(5):

print("heis")

print(i)

o/p: heis 0 1 2 3 4

2) range (start, stop)

syn:- for var-name in range(start, stop):

↓
= 2 logic.
remaining stmt.

for i in range(11, 16):

9 → 11 12 13 14 15 16
0 1 2 3 4 5

↓ Print(i)

↓ Print(i)

11 12 13 14 15

3) range (start, stop, step)

→ syn:- for var-name in range(start, stop, step):

↓
= 3 logic
remaining stmt.

ex for i in range(2, 11, +2):

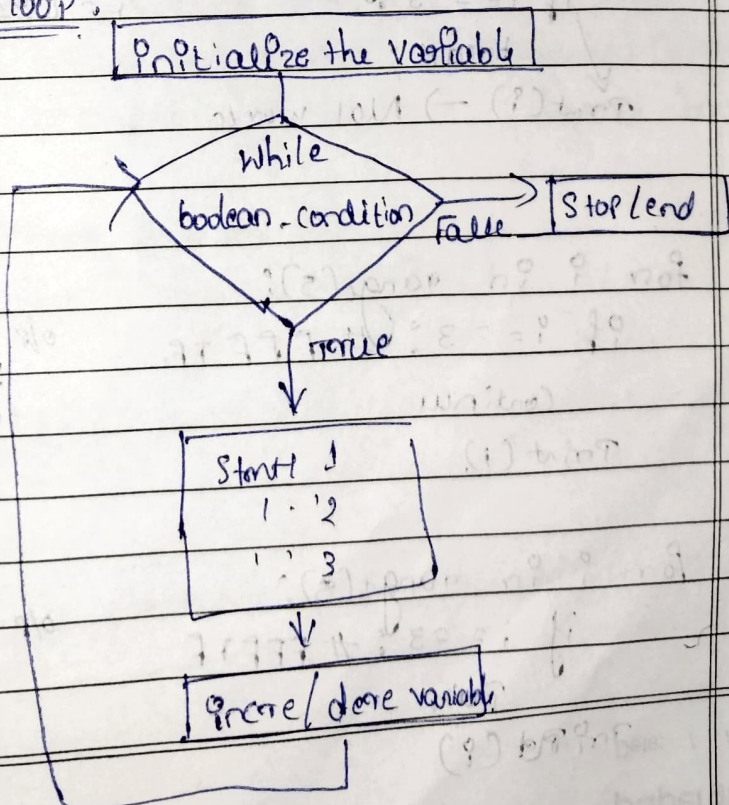
↓ Print(i)

Print(i)

i = 2 3 4 5 6 7 8 9 10 11
↑ ↑ ↑ ↑ ↑

i = 2 3 4 5 6 7 8 9 10 11
↑ ↑ ↑ ↑ ↑

⇒ while loop:



syn - Initialize a var

while (boolean condition):

 # increment/decrement the variable value

 # remaining stmt.

ex - $i = 0$

while ($i \leq 4$):

 Print("Python")

$i = i + 1$

Print(i)

Jumping Statement:

- 1) break
- 2) continue
- 3) pass

For i in range(5):

 if $i == 3$:

 break

Print(i) → Not work

for i in range(5):

 if $i == 3$: # F F F T F

 continue

Print(i)

For i in range(5):

 if $i == 3$: # F F F T F

 pass

Print(i)

Variable

1) → a variable can always start with underscore (-) or character (name)

ex `_name` or `name_`

2) A variable doesnot contain a special character except underscore (-)

ex `! % # @ ...`

3) A variable cannot start with a digit

ex `1name`, `2name` ✗

`name1`, `name2` ✓

4) A variable cannot contain keyword

ex `self`, `break`, `if`, `elif`, `else`, `for`, `ran`, `continue`, `return`, `def` etc...

5) A variable can't contain a space.

ex `Number of blades` ✗

1) `NumberOfBlades` ✓

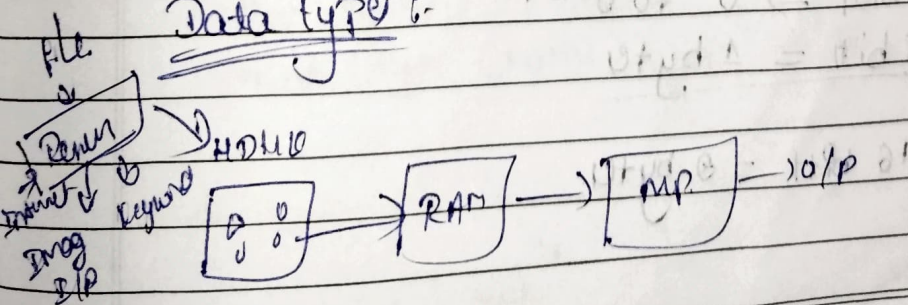
2) `Number of blades` ✓

→ Interview
Camel Convention on
Camel Concatenation

EN `import keyword`

`Print ("keyword call")`

Data type



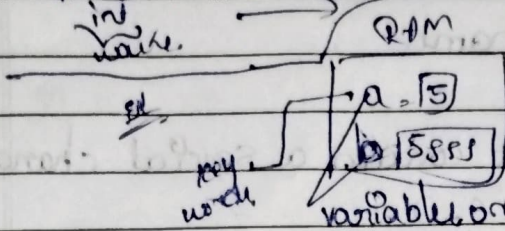
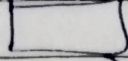
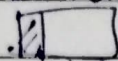
No declaration of fixed memory
Python Program

a = 5

print(a) → 5

2 = byte →

← 2 byte →



Python Program

Include <stdio.h>

void main()

int a = 5;

printf("a = %d", a);

bit & byte

1 bit → 0 or 1

2 bits → 0 0, 0 1, 1 0, 1 1

0 0 → 0 to 2² - 1

0 1 → 0 to 4 - 1

1 0 → 0 to 3 - 1

1 1 → 0 to 2³ - 1

3 bits → 0 0 0, 0 0 1, 0 1 0, 0 1 1, 1 0 0, 1 0 1, 1 1 0, 1 1 1

0 0 0 → 0 to 2³ - 1

0 0 1 → 0 to 8 - 1

0 1 0 → 0 to 7 - 1

0 1 1 → 0 to 6 - 1

1 0 0 → 0 to 5 - 1

1 0 1 → 0 to 4 - 1

1 1 0 → 0 to 3 - 1

1 1 1 → 0 to 2 - 1

4 bits → 0 to 2⁴ - 1

5 bits → 0 to 2⁵ - 1

8 bits → 0 to 2⁸ - 1

8 bits = 1 byte

16 bits = 2 bytes

Data types:

1) Integer $\rightarrow$ int $\rightarrow$ whole number

ex: $a = 10$

2) decimal value $\rightarrow$ float $\rightarrow$ decimal digit

ex: $a = 10.5$

3) string $\rightarrow$ str $\rightarrow$ collection of characters

ex: $a = \text{"Rama"}$

4) sequence datatypes $\rightarrow$

* List $\rightarrow$ [10, 20, "rama", 30] $\Rightarrow$ mutable

* Tuple $\rightarrow$ (10, 20, "rama", 30) $\Rightarrow$ immutable

* Set $\rightarrow$ {10, 20, 30} $\Rightarrow$ unordered collection $\Rightarrow$ mutable

$\rightarrow$ Frozen set $\rightarrow$ frozen(10, 20, 30) $\Rightarrow$ immutable

$\rightarrow$ dictionary $\rightarrow$ key value pairs $\Rightarrow$ dict

ex: "name": "raju"

$\rightarrow$ data types are nothing but specify the what kind of a value a variable holds

* $a = 10$

$\text{print}(a) \rightarrow 10$

$\text{print}(\text{type}(a)) \rightarrow \langle \text{class 'int'} \rangle$

* $b = 20.5$

$\text{print}(b) \rightarrow 20.5$

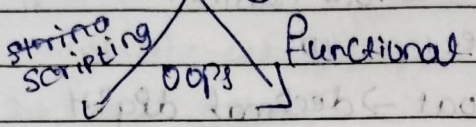
$\text{print}(\text{type}(b)) \rightarrow \langle \text{class 'float'} \rangle$

* $\text{name} = \text{"sneha"}$

$\text{print}(\text{name}) \rightarrow \text{sneha}$

$\text{print}(\text{type}(\text{name})) \rightarrow \langle \text{class 'str'} \rangle$

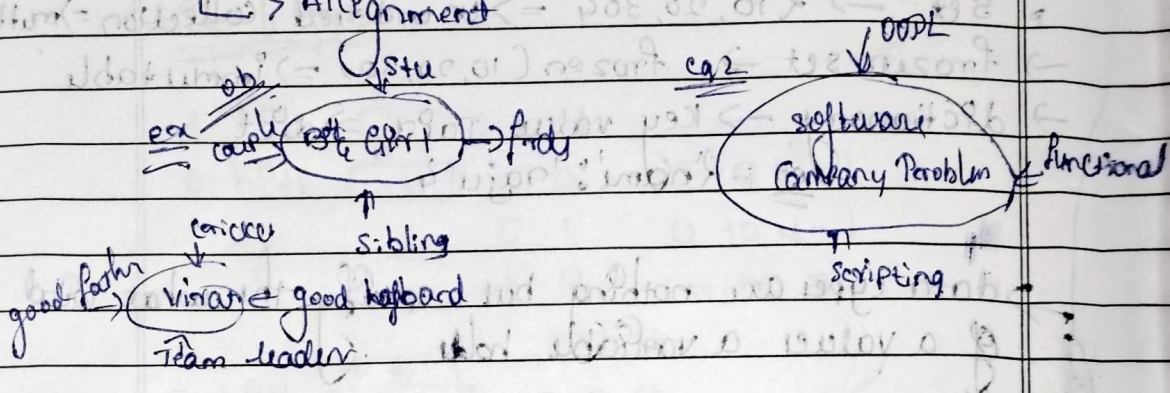
Python is language.



Object Oriented programming System

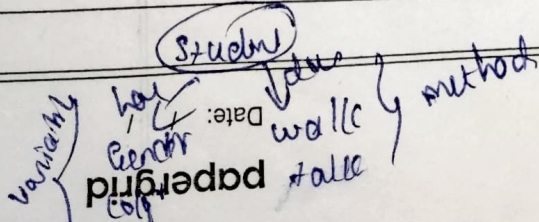
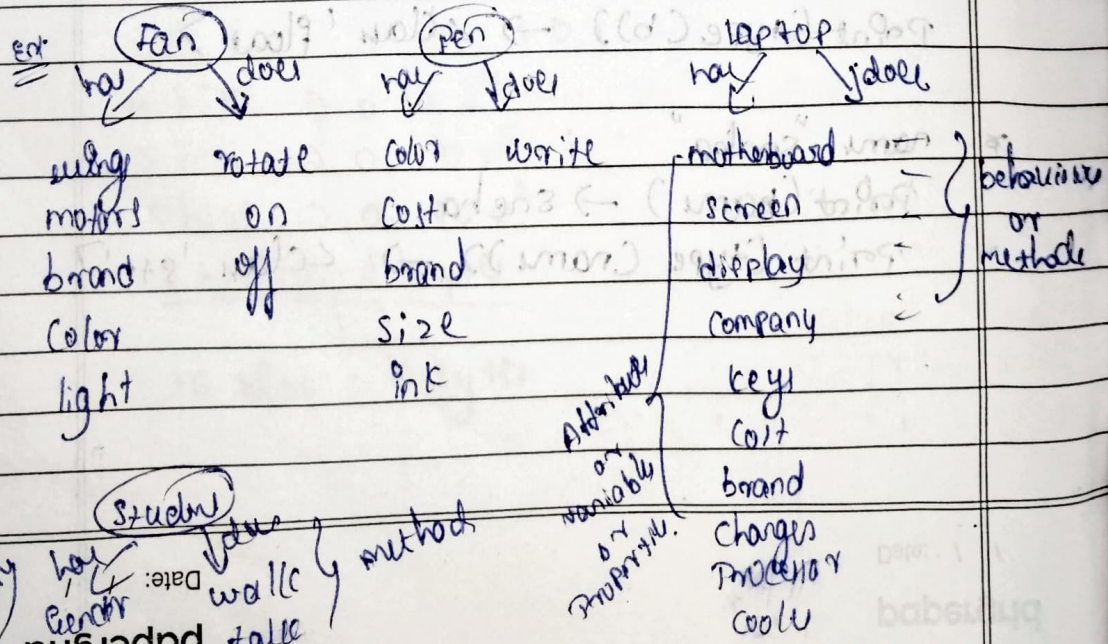
Orientation

- Pointing view
- Perspective
- Alignment

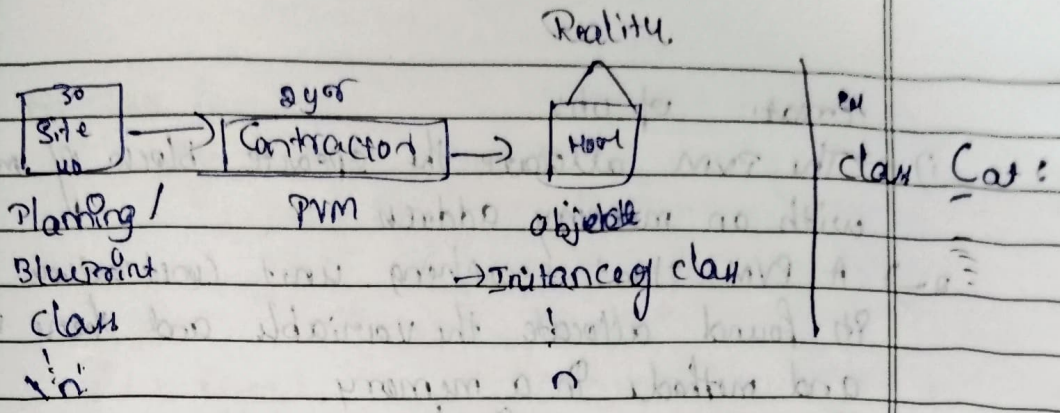


3 Steps

- 1) define a word is object or not
- 2) define type the object
- 3) classify what is has and it does



Attribute
or
variable
or
property



eg 1:

class Dog:

breed = "lab"

color = "white"

gender = "Female"

cost = 10,000

def bark():

Print("Dog is barking")

def sleep():

Print("Dog is sleeping")

Dog

has breed color gender (properties or variables)

does bark sleep drink (methods)

High Cohesional program

Proper python

eg 2

class Demo:

a = 10

b = 20

def func1():

print("display")

def func2():

Print("Hello")

Low Cohesional program

eg 3

class Fan:

def __init__(self):

self.brand = "ustd"

self.cost = 1900

self.color = "white"

self.NoOfBlades = 3

def on(self):

print("Fan is On")

def rotate(self):

Print("Fan is rotating")

Fan

has brand color No. of blades (properties or variables)

does on rotate (methods)

Fan

PA = Fan()

Print(PA.brand) ~ ustd

Print(PA.cost) ~ 1900

Print(PA.color) ~ white

Print(PA.NoOfBlades)

PA.on()

PA.rotate()

PA.off()

creat. of obj

- 1) → The PVM allocate the separate block of memory with an memory address
- 2) → A PVM start searching unit construction once it found allocate the variable and their value and methods in a memory.
- 3) The reference variable store address of an object stored in a reference variable.

Organization of RAM

code segment	→ whole code
static	→ static variable
stack	→ local variable / Reference variable / Activation record
Heap	→ object / Instance variable / Instance method

class Student:

def __init__(self):

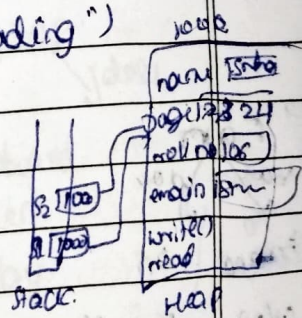
Instance variable { self.name = "sneha"
self.age = 23
self.rollno = 06

Instance method { def write(self):
print("sneha is writing")
def read(self):
print("sneha is reading")

reference variable s1 = Student()
Print(s1.name) → sneha
Print(s1.age) → 23

local variable s2 = s1

Print(s1) → address of an object
Print(s2) → address of an object
s1.email = "snehagovila@gmail.com"
Print(s1.email) → s -



class Fan:

```
def __init__(self):  
    self.brand = "Usha"  
    self.cost = 1500  
    self.colour = "brown"
```

F1 = Fan()

Print(F1) $\rightarrow$ Addressing of an obj

F2 = F1

F3 = F2

Print(id(F1)) $\rightarrow$ 1234

Print(id(F2)) $\rightarrow$ 1234

Print(id(F3)) $\rightarrow$ 1234

Print(F1 is F2) $\rightarrow$ True

Print(F2 is F3) $\rightarrow$ True

* a = 10

b = 10

c = 10

Print(a)

Print(b)

Print(c)

Print(id(a)) $\rightarrow$ 1234

Print(id(b)) $\rightarrow$ 1234

Print(id(c)) $\rightarrow$ 1234

Print("a is b") $\rightarrow$ True

Print("c is b") $\rightarrow$ True

d = 20

e = 20

Print(id(d)) $\rightarrow$ 567

Print(id(e)) $\rightarrow$ 567

Print(d is e) $\rightarrow$ True

Print(a is e) $\rightarrow$ False

